

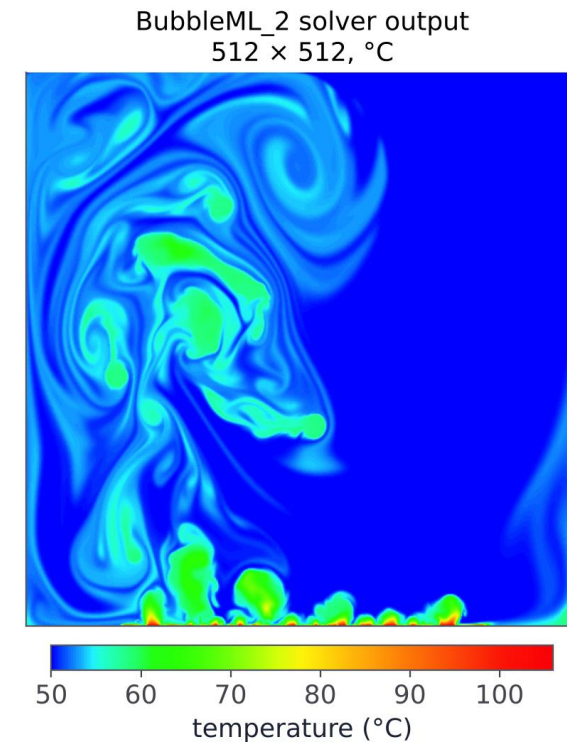
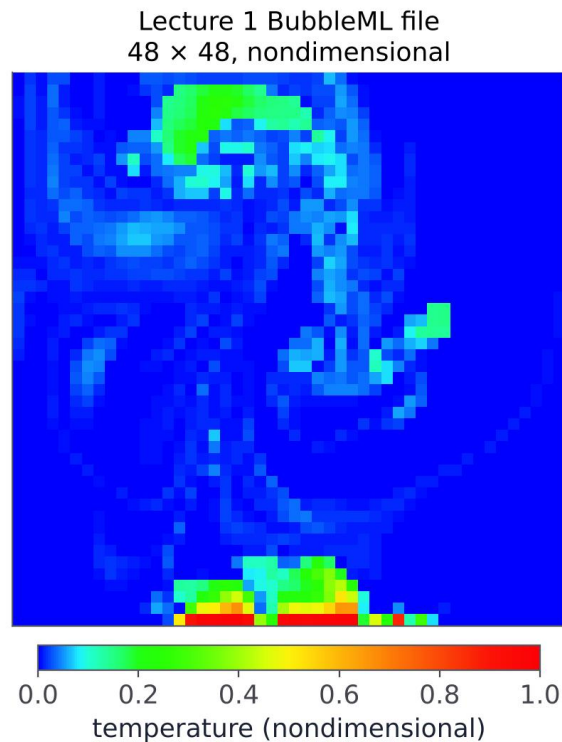
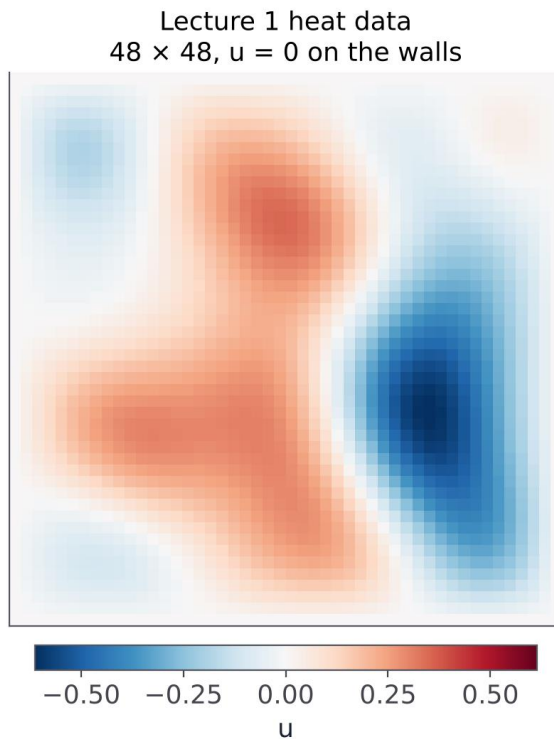


ML FOR SCIENCE · LECTURE 4

Data Representations for SciML

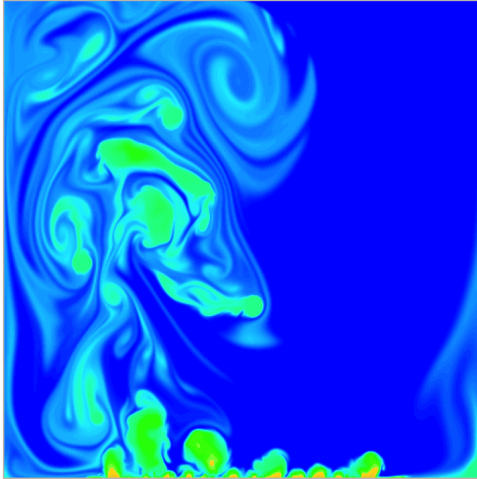
From solver output to model input

- Lecture 1 trained SmallCNN on **48 x 48 arrays**: one channel for the heat equation, three for BubbleML (temperature, x-velocity, y-velocity)
- The solver computes far more: the BubbleML_2 run on the right has **512 x 512** cells, saved every 0.1 time units
- Lecture 3: the solver's choices end up in the data. Today: our choices of *how to store* the data end up in the model

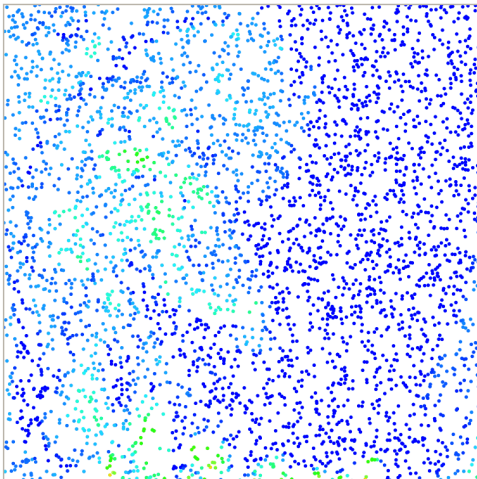


One field, many representations

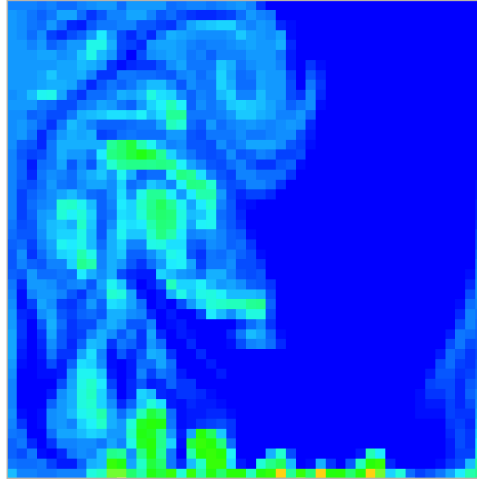
grid: 512×512
262,144 cell values



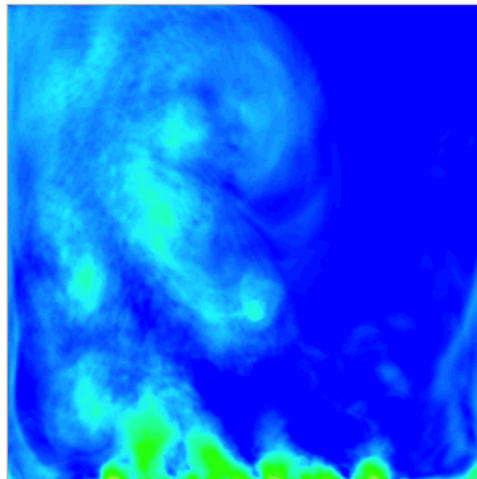
point cloud at random positions
4,000 points, each storing (x, y, T)



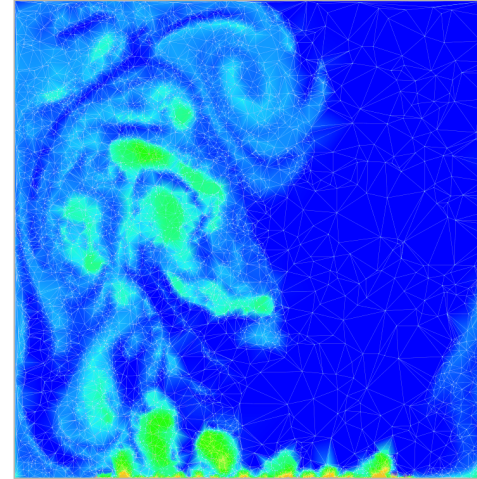
coarser grid: 48×48
2,304 cell averages



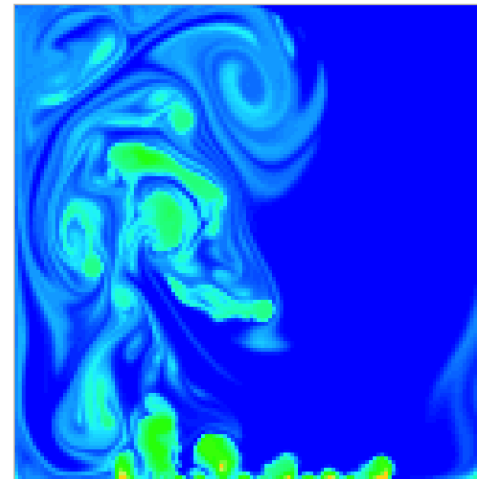
POD basis of 256×256 modes
20 coefficients + the modes



triangle mesh, refined where T varies
6,004 vertex values + connectivity



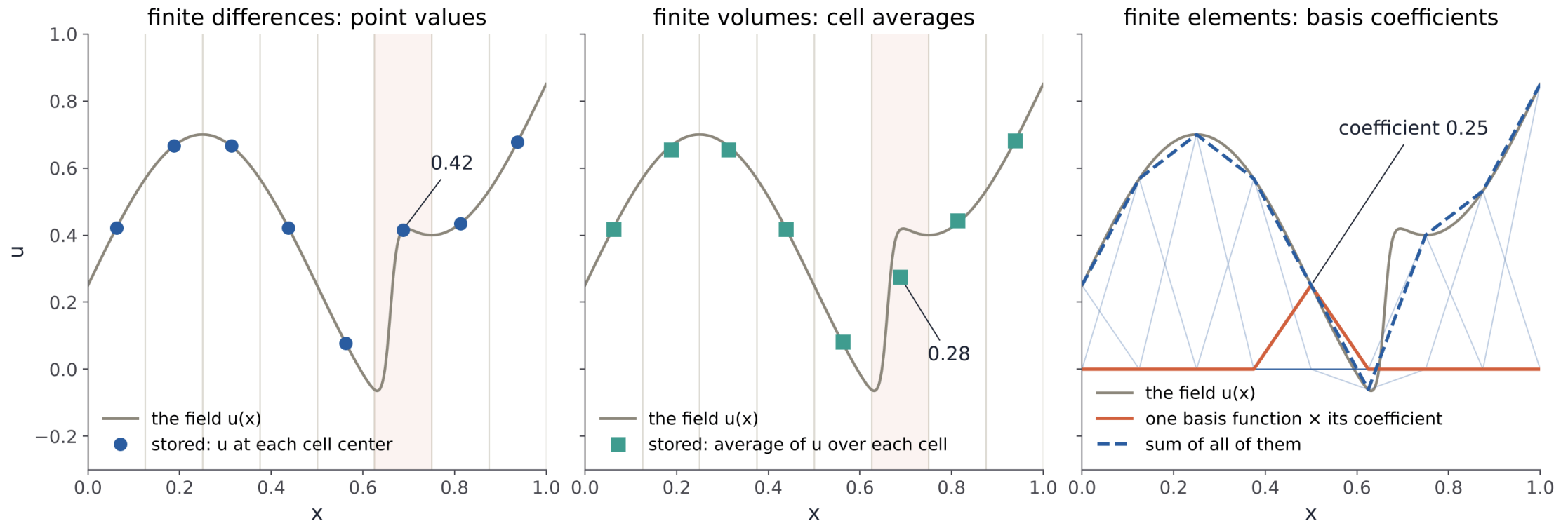
neural field: a network $T(x, y)$
263,425 weights, fit at 128×128



- Same temperature frame, stored six ways
- They differ in **stored value** (e.g., cell values, vertex values, coefficients, network weights), **number of values** stored, and **structure** retained
- Each representation suits a different kind of model:
 - **grids**: CNNs, U-Nets
 - **meshes and point clouds**: graph networks
 - **coefficients**: linear models, Fourier layers
 - **neural fields**: coordinate networks

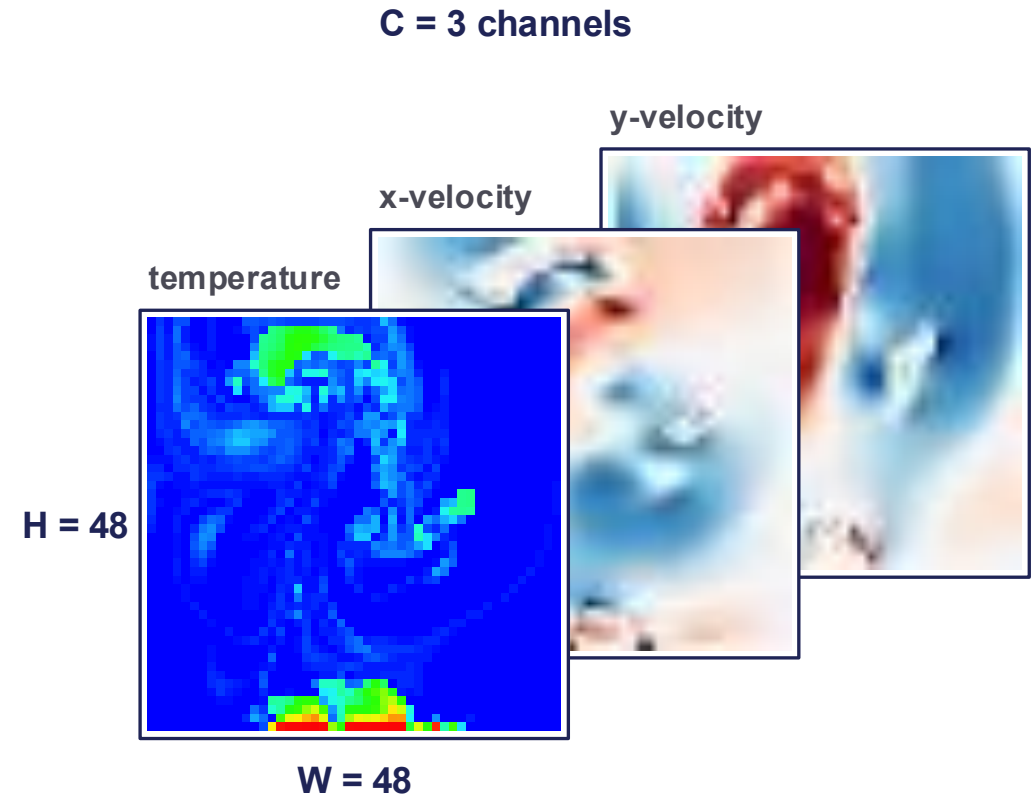
What does a stored value represent?

- The PDE discretization methods from Lecture 3 store one value per cell or node, but the values mean different things:
 - point value**: u at the cell center. Finite differences
 - cell average**: the average of u over the cell. Finite volumes, which conserve mass and energy exactly (e.g., Flash-X, the BubbleML solver, is a finite-volume code)
 - coefficient**: the weight of a basis function; $u \approx \text{sum of basis functions} \times \text{coefficients}$. Finite elements
- Point values and averages differ near sharp features (**0.42 vs. 0.28** in the shaded cell). This matters when you **downsample, interpolate or combine datasets**



A grid sample is a tensor

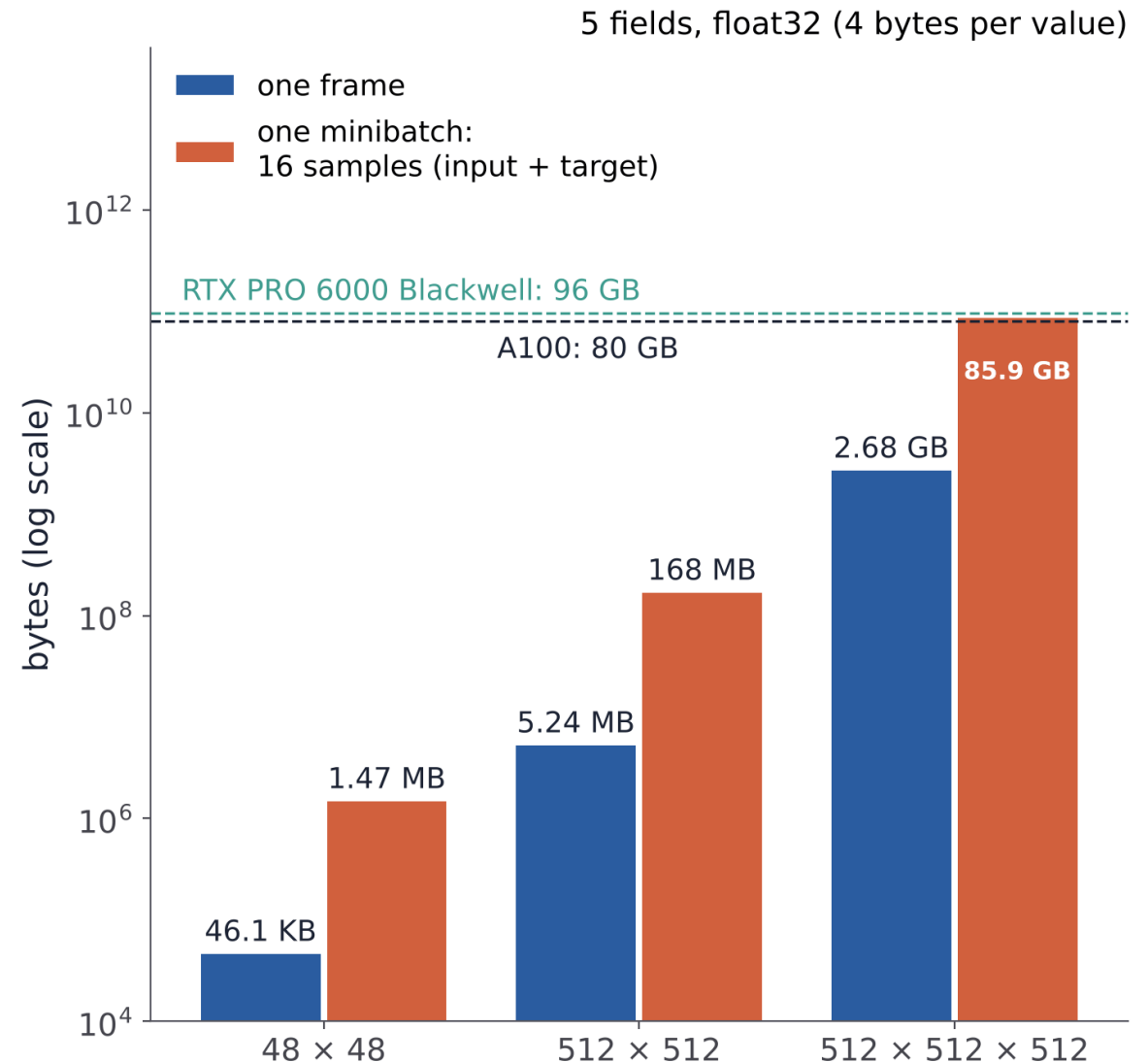
- A 2D field on an $H \times W$ grid is an $H \times W$ array; several fields stack as **channels**
- A minibatch has shape **[B, C, H, W]**, the layout `nn.Conv2d` expects
 - Lecture 1 BubbleML: [16, 3, 48, 48]: 16 samples, channels temperature, x-velocity, y-velocity
 - Lecture 1 heat data was stored as (trajectory, time, y, x) = (100, 101, 48, 48) and reshaped to [B, 1, 48, 48]
- k past frames as input: stack them as channels, $C \cdot k$
- 3D fields add a depth axis: **[B, C, D, H, W]**
- Unlike the red, green and blue of a photo, each channel is a different physical quantity with its own units and scale



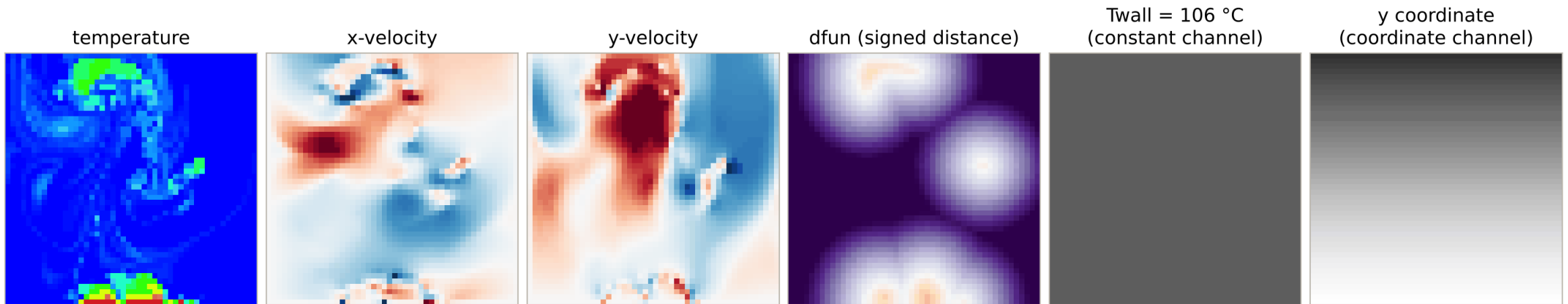
one sample: $[C, H, W] = [3, 48, 48]$

Scientific grids are large

- One 512 x 512 frame of 5 float32 fields (temperature, velx, vely, pressure, dfun): 5.2 MB
- One simulation, 2,001 frames: 10.5 GB. In 3D at 512³: **5.4 TB**
- So we read one window of frames at a time from disk
 - Lecture 1's HDF5Dataset does this: `self.data[k][idx]` reads one frame
- Or we **store less**: coarser grids, fewer frames, compressed representations
- What must fit on the GPU is one minibatch plus the network's activations
 - 2D: 16 samples take 0.17 GB. 3D: **86 GB**, more than A100 memory
- Training also stores every layer's output for backpropagation (Lecture 2). In 3D, one 64 channel layer for a single sample is 34 GB, so even a *batch of one won't fit*



Parameters, coordinates, and geometry as input channels



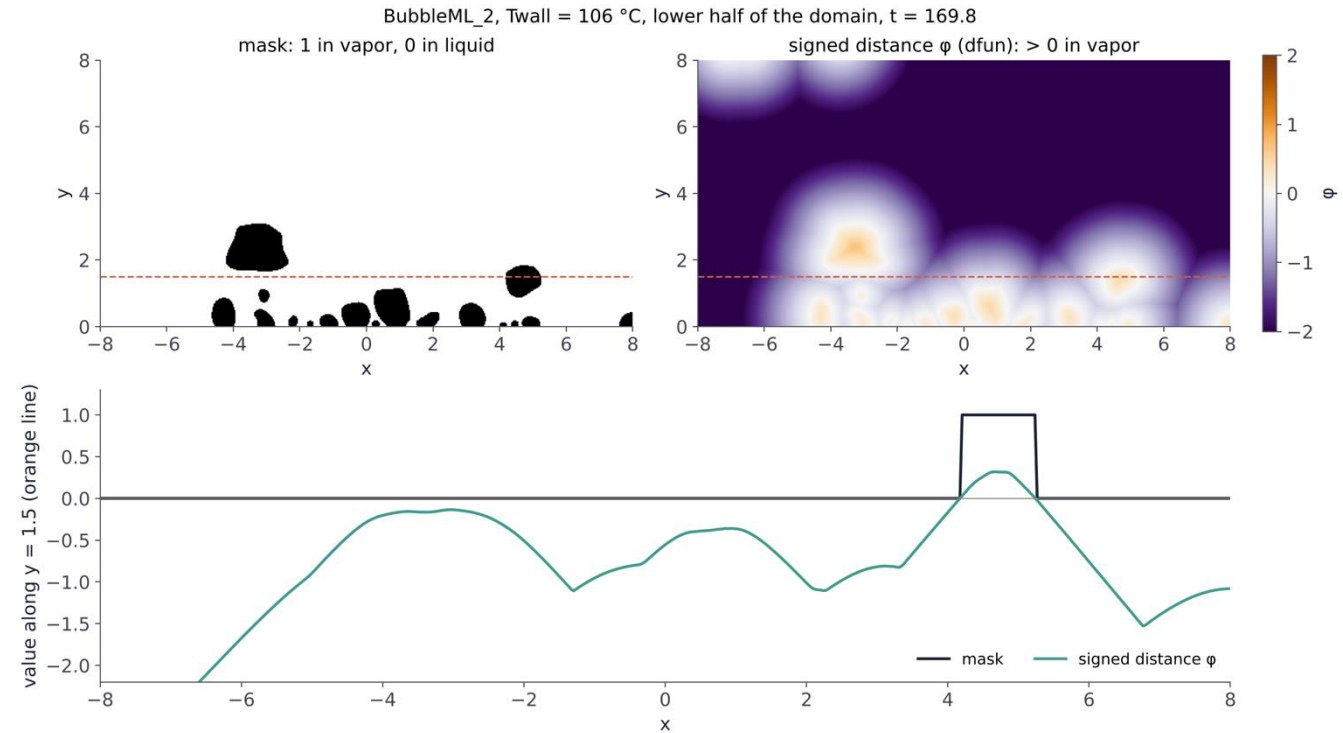
- The Lecture 1 BubbleML model had no input for the wall temperature; runs at 100 and 106 °C used the same three channels
- **Parameters** such as T_{wall} or the Reynolds number can be added as a constant channel, with the same value in every cell
- **Coordinates** can be added as channels. A convolution applies the same weights at every position, so position information must come from the input (CoordConv, Liu et al. 2018)
 - here the heater is at $y = 0$, and a y channel gives each cell its height above the heater
- **Geometry** can be added as a mask or a signed distance (next slide)

Geometry as a mask or a signed distance function

- **Mask:** 1 in vapor, 0 in liquid, with a jump at every interface
- **Signed distance Φ :** distance to the nearest interface, positive in vapor and negative in liquid (BubbleML's dfun)

$$\phi(\mathbf{x}) = \pm \text{dist}(\mathbf{x}, \text{surface}), \quad |\nabla \phi| = 1, \quad \mathbf{n} = \nabla \phi$$

- The interface is the set where $\Phi = 0$. Near it $|\nabla \Phi| = 1$ and $\nabla \Phi$ is the interface normal
- Lecture 3: a jump needs many Fourier modes. Φ is continuous, with kinks only midway between bubbles
- Lecture 1, Exercise 1 added dfun as a fourth input channel

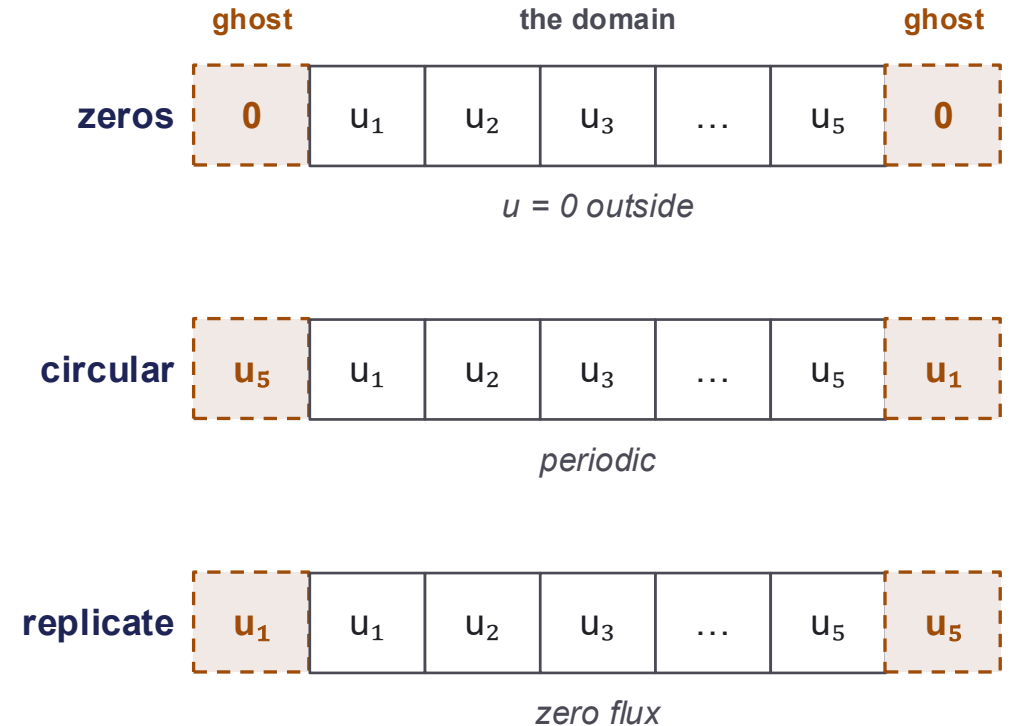


Boundary conditions live in the padding

- Lecture 3: the Laplacian stencil is a 3 x 3 convolution

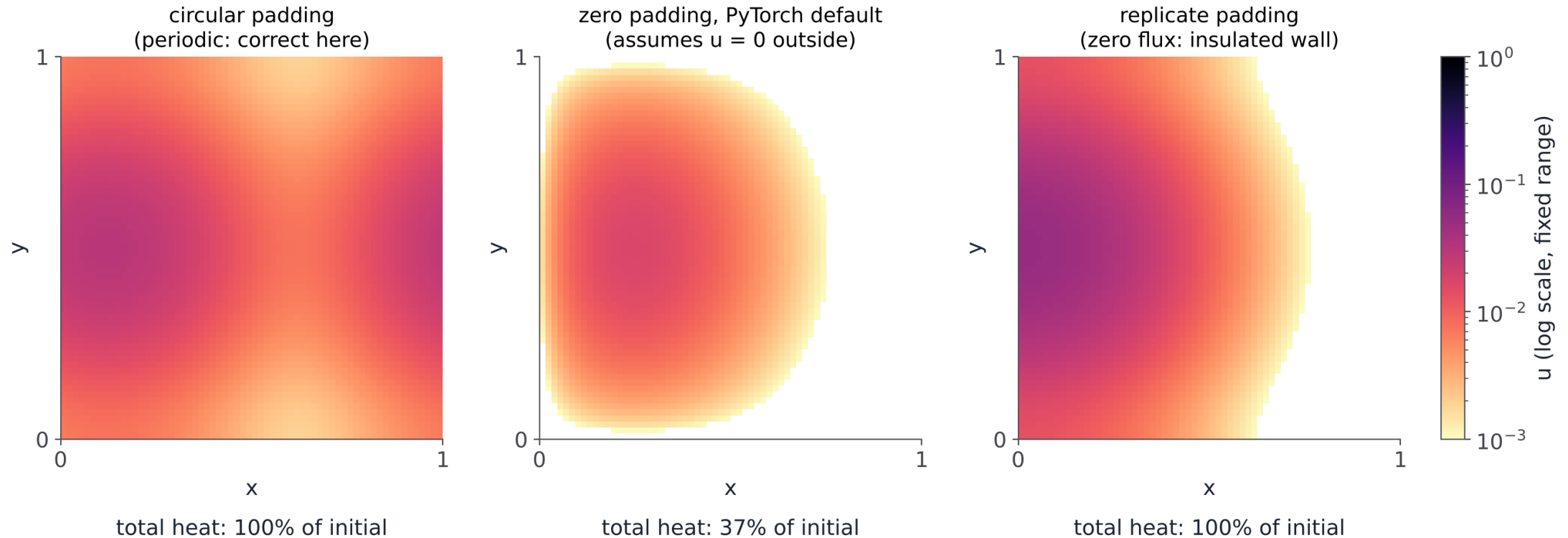
$$(\nabla^2 u)_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{\Delta x^2}$$

- At a wall the stencil needs values outside the domain. A solver sets these **ghost cells** from the boundary condition
- A CNN sets them by **padding**, so each padding mode corresponds to a boundary condition:
 - zeros**: $u = 0$ outside the domain. PyTorch's default, used by SmallCNN
 - circular**: values wrap around, a periodic boundary
 - replicate**: copies the edge value, zero flux (insulated wall)



Every padded convolution applies a boundary condition. Choose the one that matches the physics.

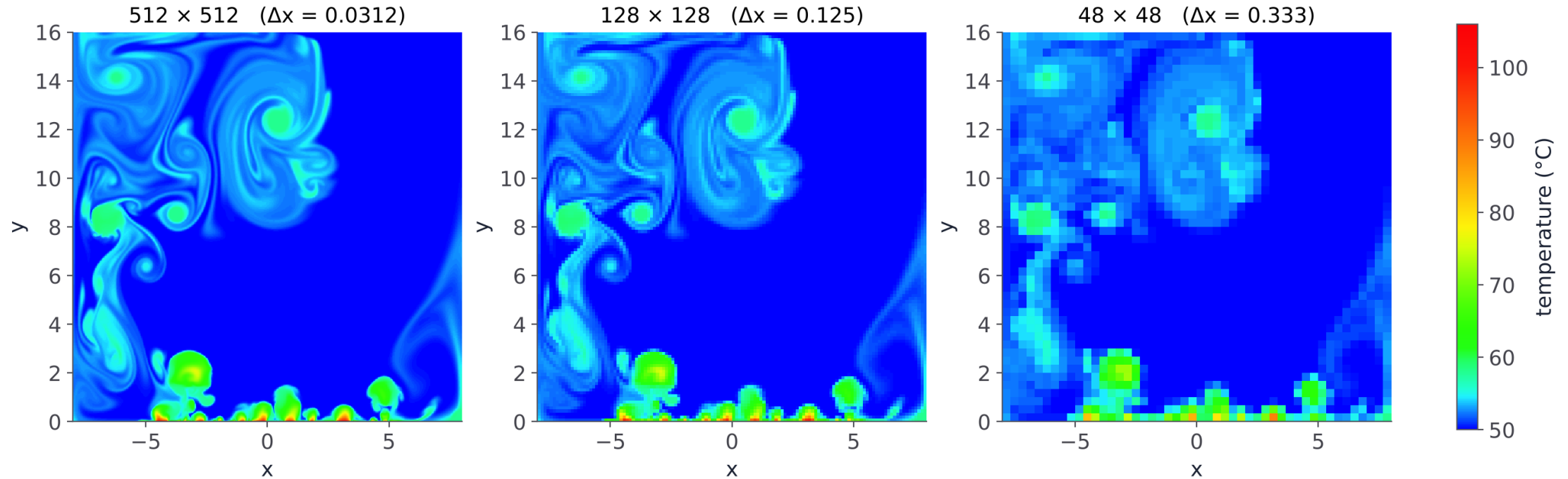
Wrong padding is a wrong boundary condition



- Same stencil, time step and initial bump near the left wall, on a periodic domain. Only the padding differs.
- **Circular**: heat crosses the left wall and enters from the right.
- **Zero**: heat leaves through the walls, **37% remains** after 600 steps.
- **Replicate**: heat is conserved but stays on the left

Coarser grids lose bubbles and boundary layers

BubbleML_2, $T_{\text{wall}} = 106\text{ }^{\circ}\text{C}$, cell averages over coarser cells, $t = 169.6$



- Each coarse cell is the average of its fine cells. From 512 to 48 cells per side, Δx increases from 0.031 to 0.33
- Features smaller than a coarse cell are averaged: at 48 x 48, **small bubbles** and **thermal boundary layer** at the heater are lost, and these determine the heat transfer
- A model cannot learn structure the grid does not resolve. Choose the resolution from the **smallest feature the physics depends on**.

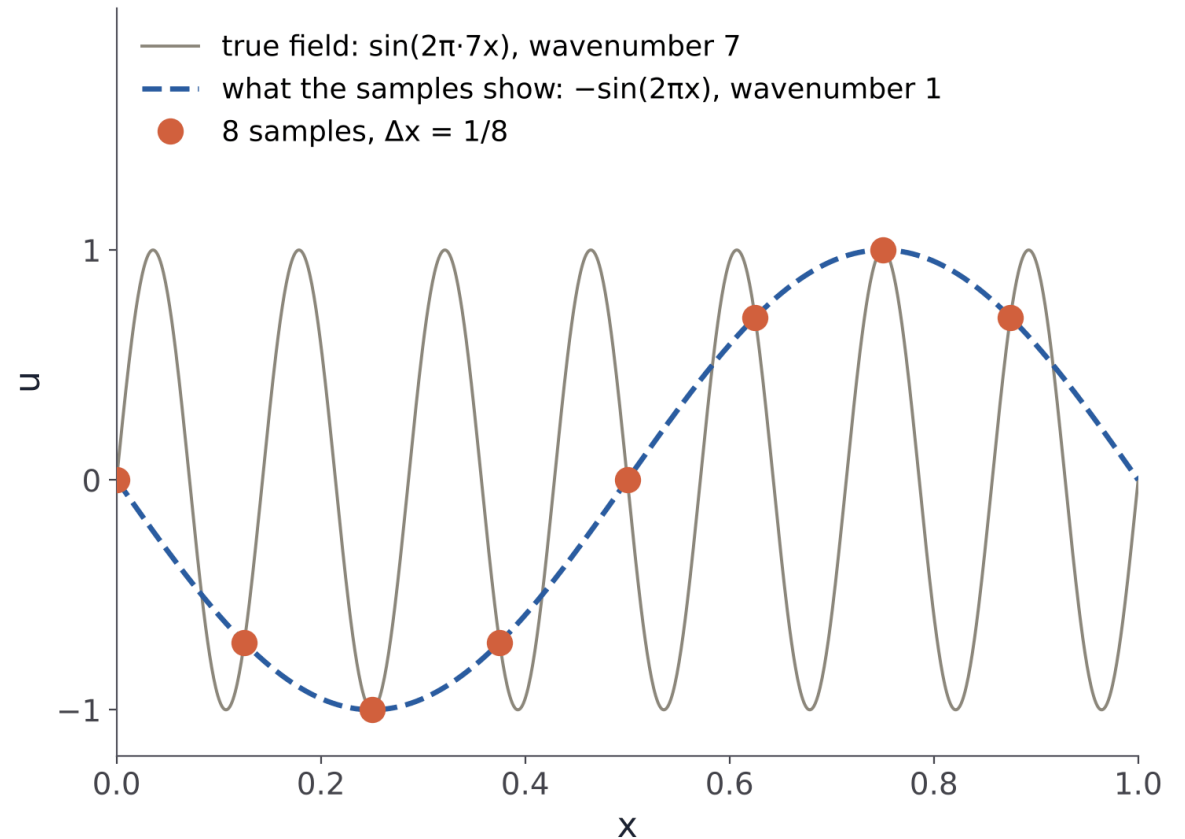
Undersampling causes aliasing

- A field on a periodic domain is a sum of Fourier modes; the wavenumber k counts waves across the domain. Large scales are low k

$$u(x) = \sum_k \hat{u}_k e^{2\pi i k x}$$

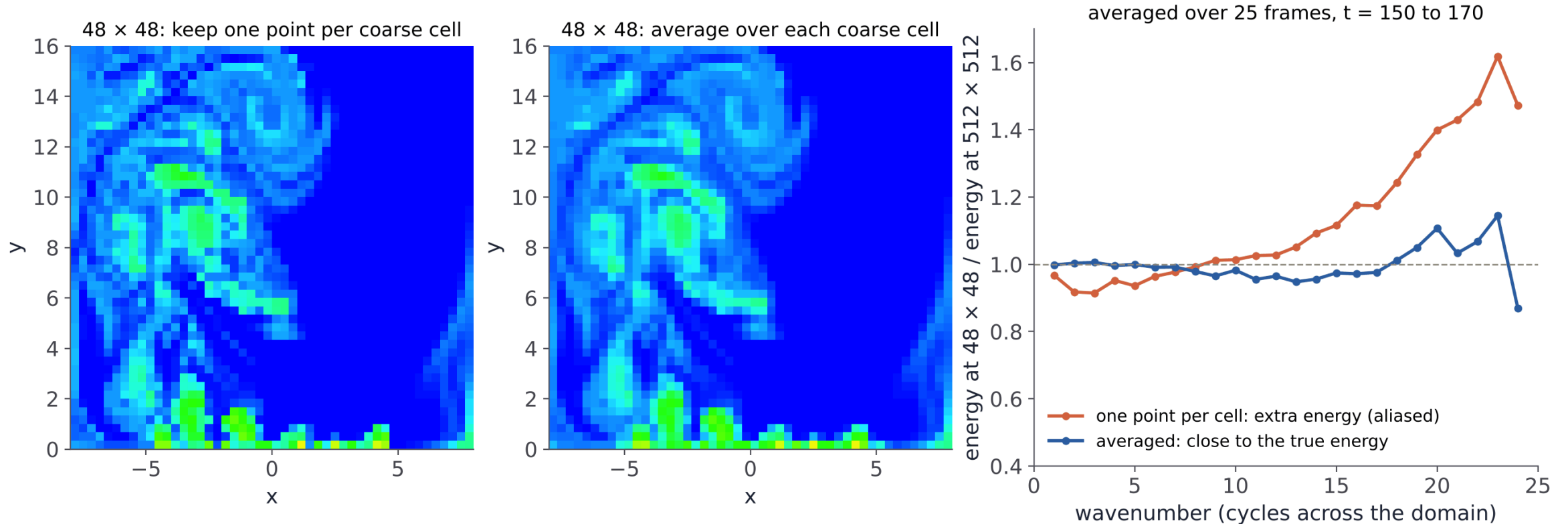
- N equally spaced samples represent wavenumbers up to $N/2$ (**Nyquist limit**)
- A higher mode appears as a lower one: **aliasing**
 - 8 samples of wavenumber 7 match wavenumber -1 exactly:

$$\sin(2\pi \cdot 7 x_j) = -\sin(2\pi x_j), \quad x_j = j/8$$



Coarse sampling can turn fine scales into **false large-scale structure**.

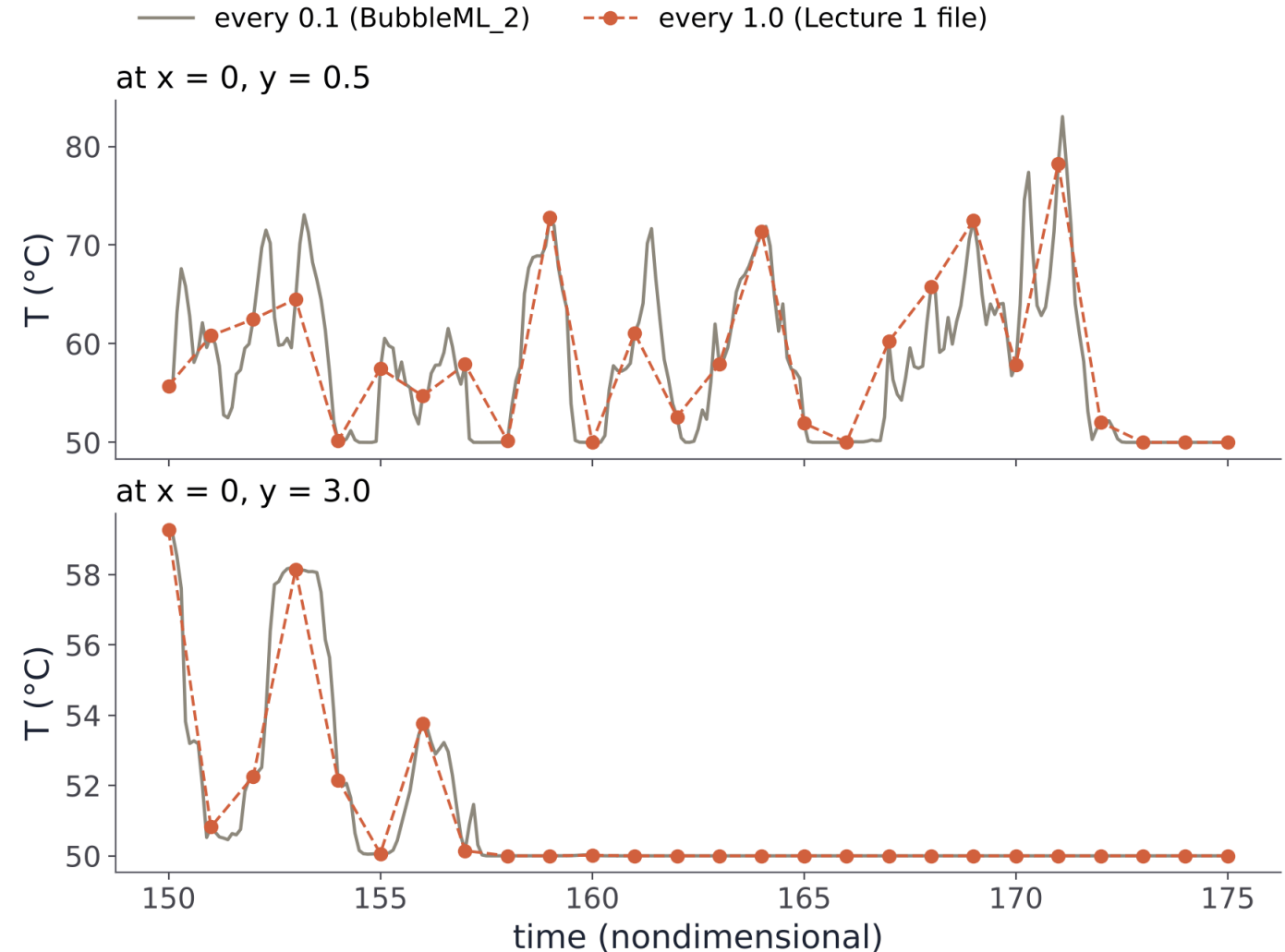
Averaging before downsampling limits aliasing



- Keeping one point per coarse cell adds **about 1.5x the true energy** at wavenumbers 20 to 24, from fine structure aliased onto the coarse grid. Averaging stays close to the true energy
- When you coarsen data for training, average (or low-pass filter) first

Sampling in time

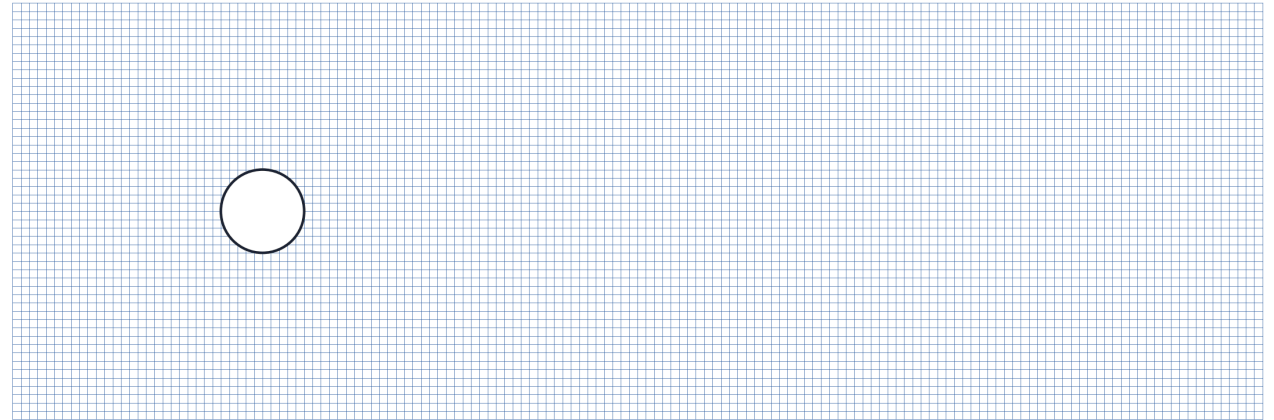
- Lecture 3: the solver steps at $\Delta t \leq 4 \times 10^{-4}$. BubbleML_2 saves every 0.1 (at least 250 solver steps); the Lecture 1 file every 1.0 (at least 2,500)
- **Wider spacing**: each training pair spans a larger, harder step (Lecture 1, Exercise 2)
- **Narrower spacing**: consecutive frames are nearly identical, and a random split leaks
- Events shorter than the spacing are **missed**: near the heater, many peaks fall between the orange points
- Spacing in time is a trade-off



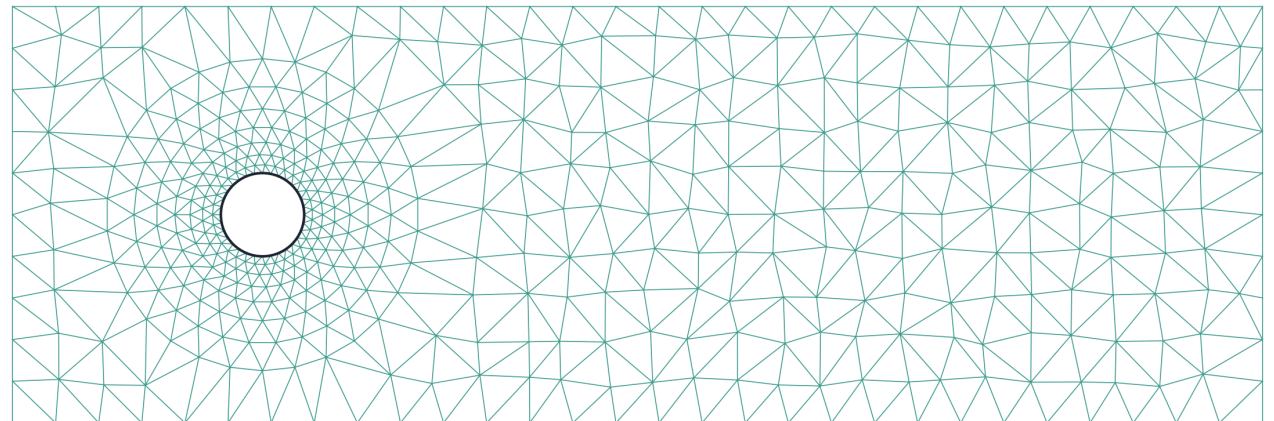
Complex geometry calls for unstructured meshes

- Flow past a cylinder needs small cells near the wall, where the velocity changes fast
- A uniform grid then needs small Δx everywhere: **~7,400 cells**
- A mesh places small cells near the wall and larger ones elsewhere: **910 triangles** for the same resolution at the wall
- Curved walls follow the mesh edges; on a grid they become staircases
- A mesh is irregular, so the **[B, C, H, W] layout** and standard convolutions no longer apply

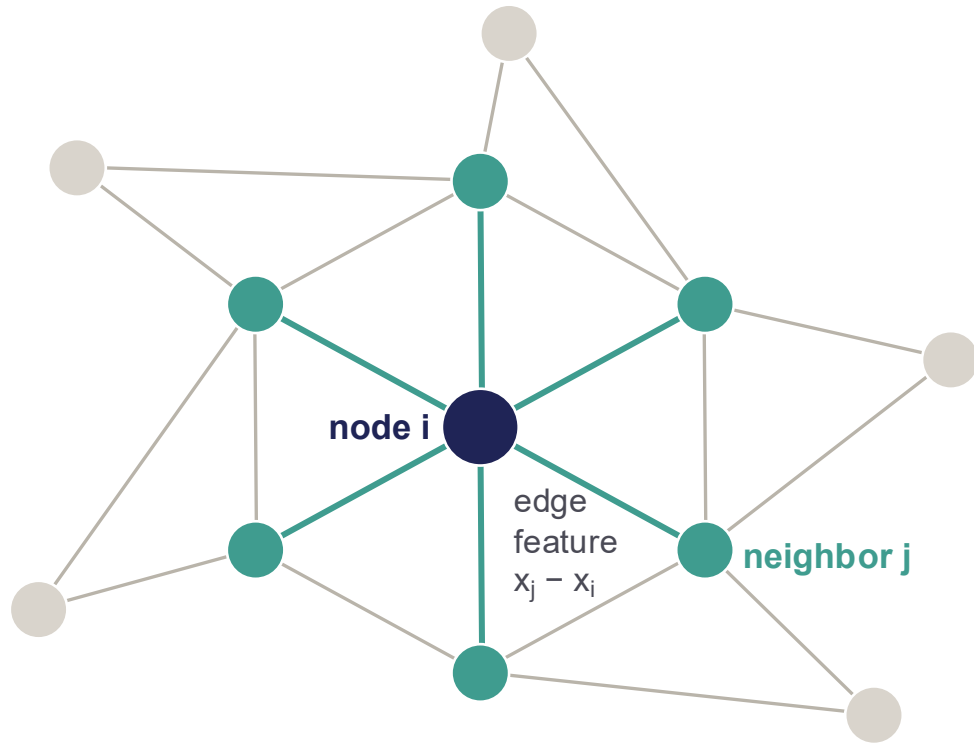
uniform grid fine enough at the cylinder: $\Delta x = 0.02$ everywhere, 7,421 cells



triangle mesh refined near the cylinder: edge length 0.02 to 0.12, 910 triangles



Meshes as graphs



stored as: node features $[N, C]$ · positions $[N, 2]$ · edge list $[2, E]$

- **Nodes**: mesh vertices or cells, each with a feature vector h_i : velocity, pressure, node type (wall, inflow, fluid)
- **Edges**: pairs of nodes that share a mesh edge, with features such as the relative position $\mathbf{x}_j - \mathbf{x}_i$ and its length
- Lecture 3: a stencil combines a point with its neighbors using fixed weights. **Message passing** combines them with learned functions:

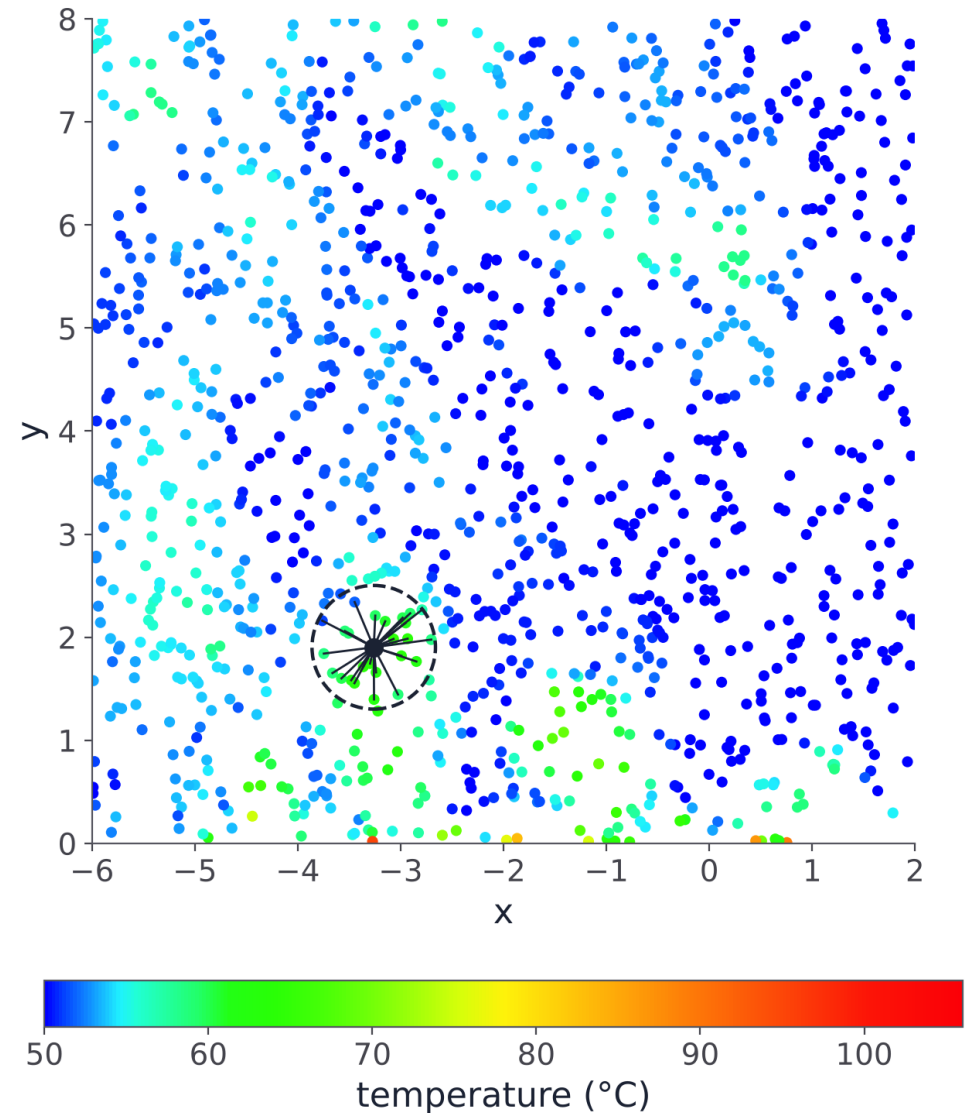
$$h'_i = \phi\left(h_i, \sum_{j \in \mathcal{N}(i)} \psi(h_i, h_j, \mathbf{x}_j - \mathbf{x}_i)\right)$$

- ψ : learned message from j to i ; ϕ : learned update; $\mathcal{N}(i)$: neighbors of i
- MeshGraphNets (Pfaff et al. 2021); Lecture 12 covers graph networks
- Molecules and crystals are graphs too, with atoms as nodes

Particles, point clouds, and sensor data

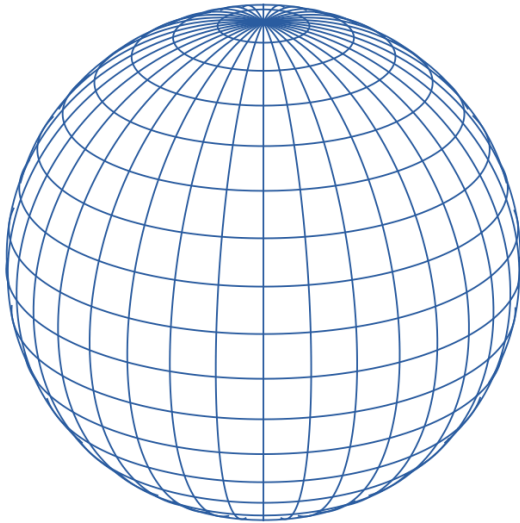
- A **point cloud** stores N positions $[N, d]$ and their features $[N, C]$, with no grid or edges
- Used when the data points move: particle fluids, molecular dynamics
- Sensor and experimental data have the same form: ocean drifters, weather stations, LiDAR scans. Often sparse, irregular in time and noisy; store a mask or an uncertainty with each value
- Neighbors are found by distance (here 25 points within $r = 0.6$) and recomputed whenever points move
- Row order is arbitrary: every ordering describes the same system, so the model output must be the same for all orderings

$$F(P X) = F(X) \quad \text{for every reordering } P$$

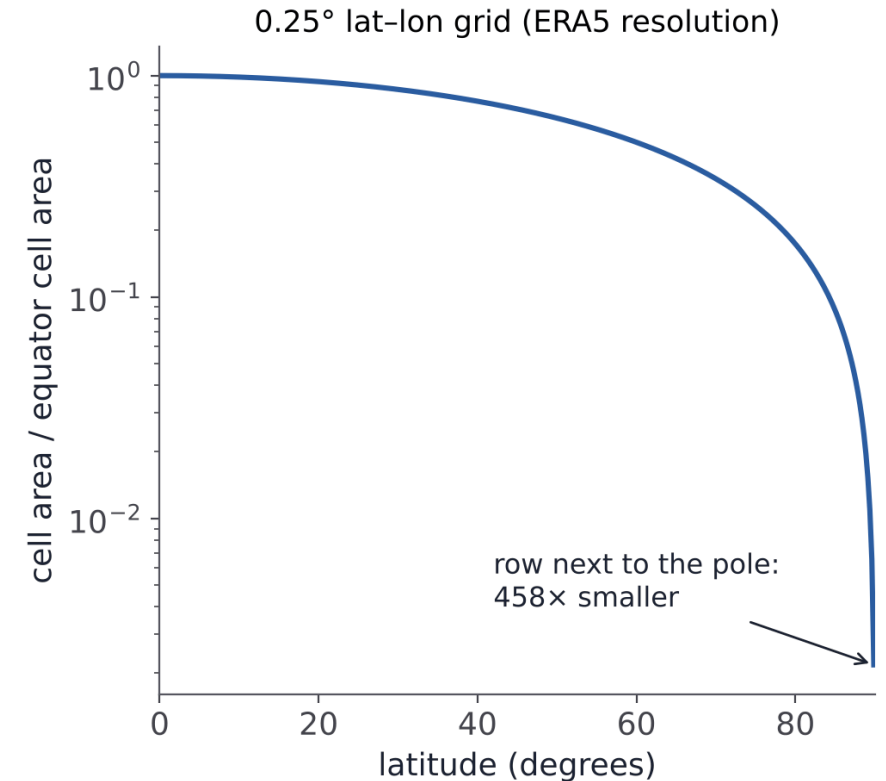
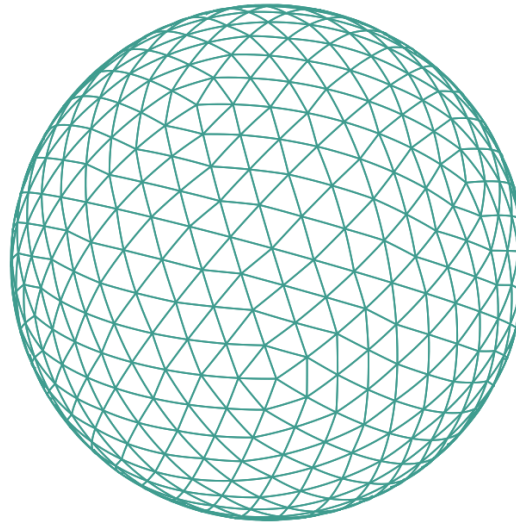


Grids on the sphere

latitude-longitude grid (10° spacing)
lines crowd together at the poles



icosahedral mesh, refined 3 times
642 nodes, nearly equal spacing



- A latitude-longitude grid is a regular array, so CNNs apply, but cells shrink toward the poles: at 0.25° (ERA5), cells next to a pole have **1/458 the area** of equator cells
- Equal steps in the array correspond to unequal distances on the sphere
- GraphCast (Lecture 1) works on an icosahedral mesh refined 6 times: **40,962 nearly evenly spaced nodes**, processed as a graph

Representing a field with a few coefficients

- A 512 x 512 frame holds 262,144 values per field, but neighboring values are strongly correlated
- Write the field as a sum of basis functions and store only the coefficients:

$$u(\mathbf{x}) \approx \bar{u}(\mathbf{x}) + \sum_{k=1}^r a_k \varphi_k(\mathbf{x})$$

- $\bar{u}$: mean field; φ_k : basis functions; a_k : coefficients
- Lecture 3: Fourier modes are one such basis. Smooth fields need few modes; a jump needs over 1,600 for 1% error
- Fourier modes are fixed in advance. POD finds the basis that needs the **fewest coefficients for a given dataset**

Recall

$$u(x) = \sum_k \hat{u}_k e^{2\pi i k x}$$

$\hat{u}_k$: coefficient of the wave with k periods across the domain

Compression is choosing a basis in which our data need few coefficients.

Proper orthogonal decomposition

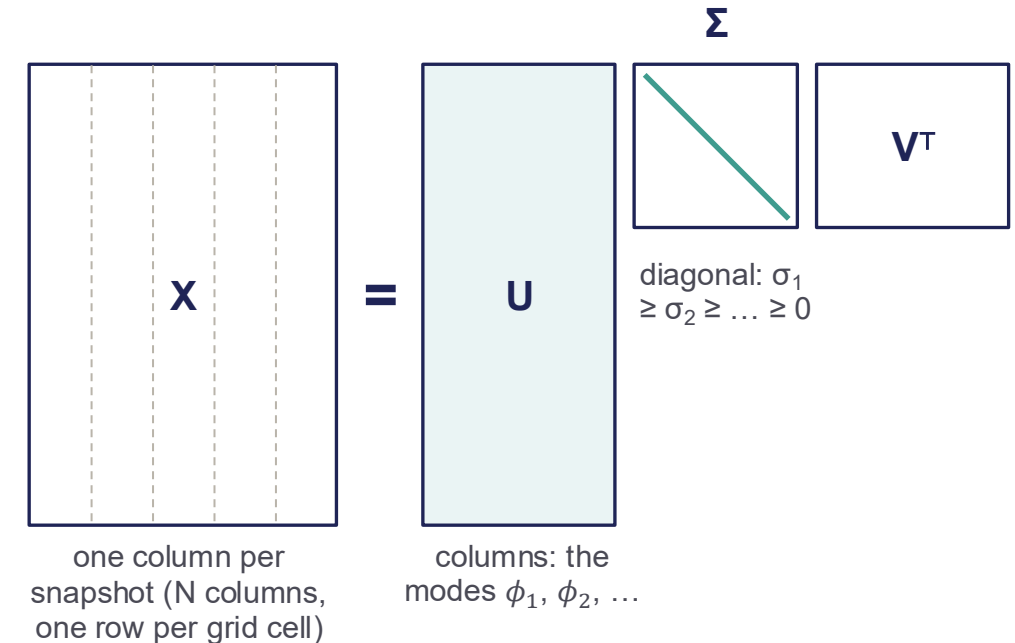
- Flatten each snapshot to a vector, subtract the mean, and place them as the columns of X :

$$X = [u_1 - \bar{u}, u_2 - \bar{u}, \dots, u_N - \bar{u}] = U \Sigma V^T$$

- The singular value decomposition gives orthonormal columns of U : the **POD modes** ϕ_k , ordered by the singular values σ_k
- σ_k^2 is the energy (squared L^2 norm) that mode k captures
- The coefficients of a snapshot are its projections onto the first r modes:

$$a = U_r^T (u - \bar{u})$$

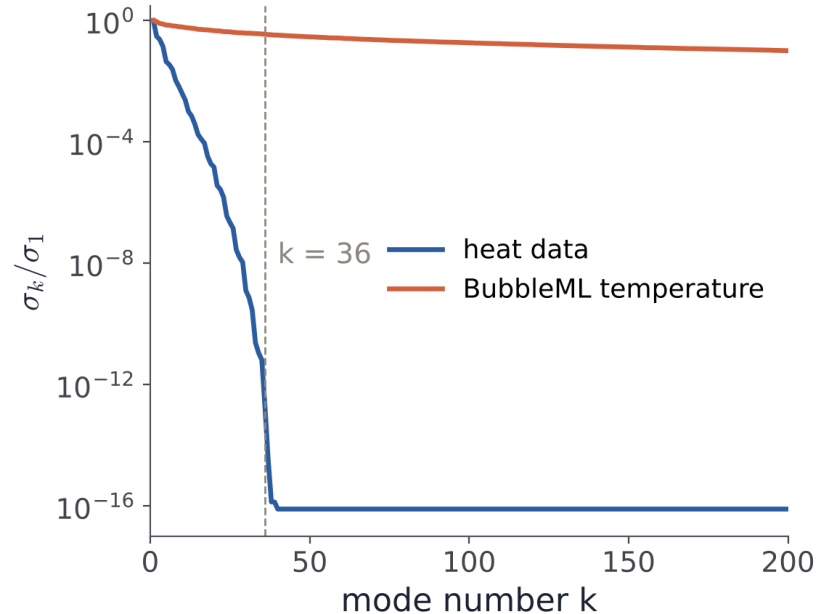
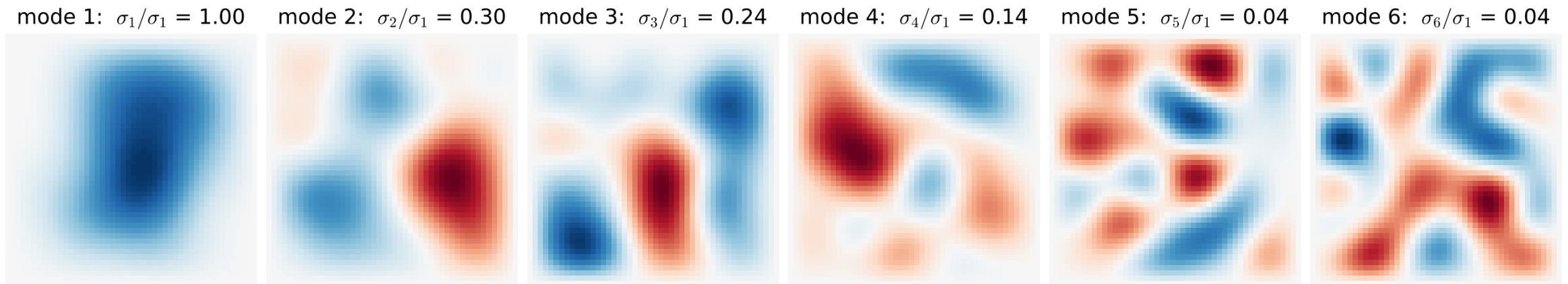
- Same computation as principal component analysis (PCA), applied to fields



Keeping r modes gives the best rank- r approximation in the L^2 norm (Eckart & Young 1936):

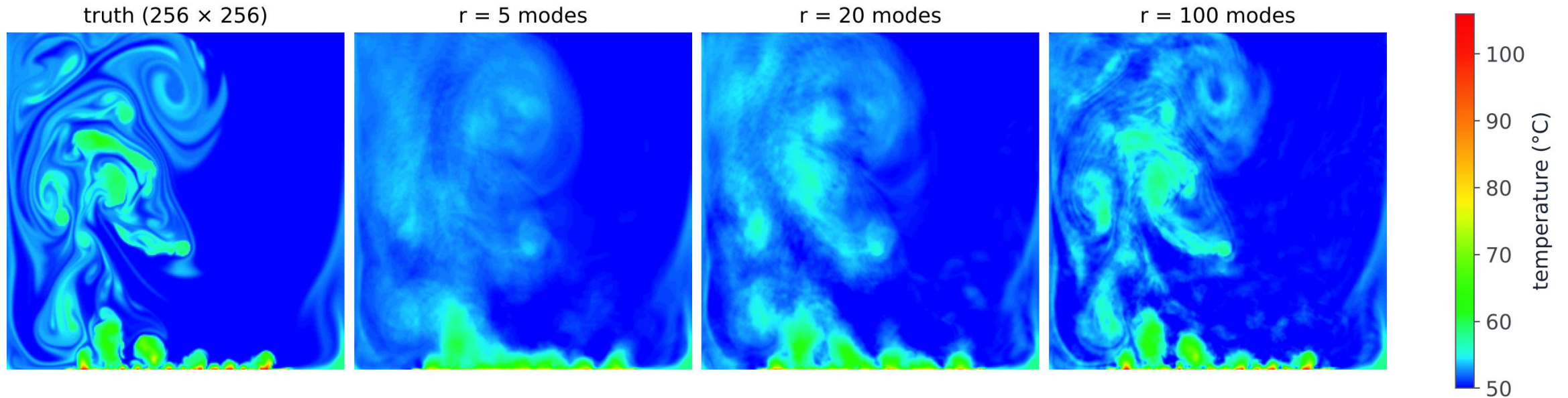
$$\min_{\text{rank } X_r = r} \|X - X_r\| = \left(\sum_{k > r} \sigma_k^2 \right)^{1/2}$$

POD of the Lecture 1 heat data



- Every heat snapshot is a sum of the same 36 sine modes ($m, n \leq 6$) with weights that decay in time, so the snapshots span at most 36 dimensions
- POD finds this: σ_k drops to round-off after **$k = 36$** ; 4 modes hold 99% of the energy
- The modes are combinations of low sines, ordered by energy in this dataset
- BubbleML temperature (orange) decays slowly: σ_{200} is still 10% of σ_1

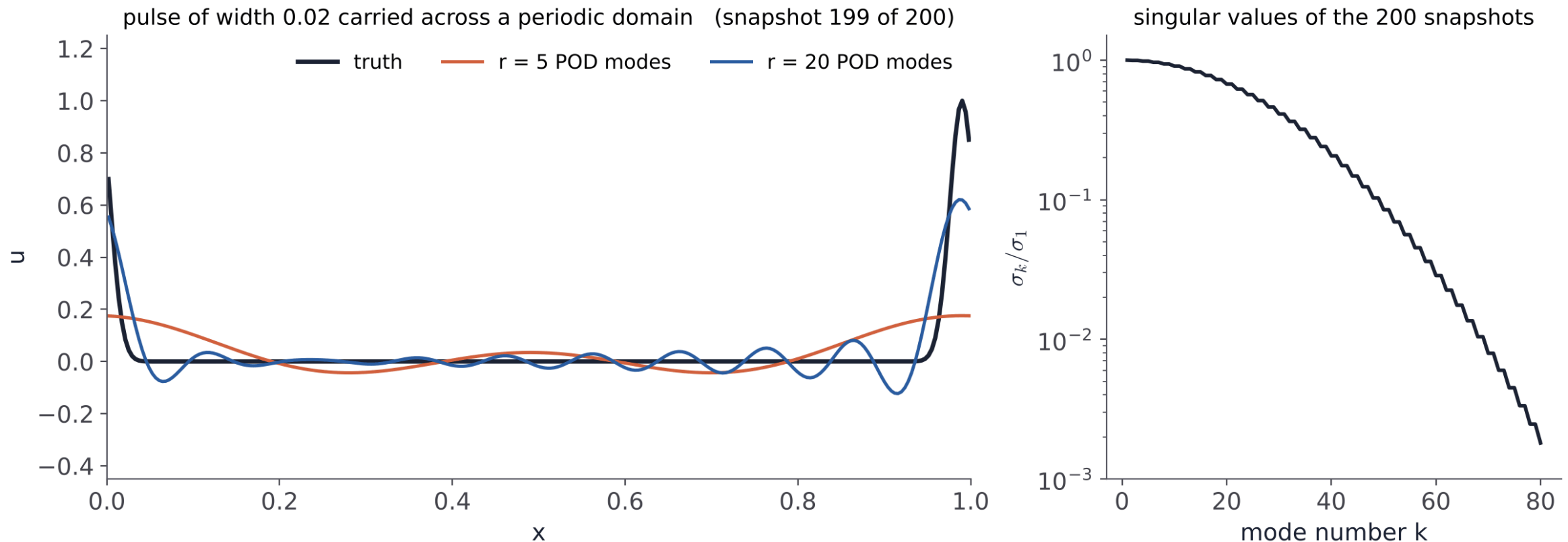
POD of BubbleML temperature



- 250 snapshots ($t = 100$ to 200 , every 0.4), 256×256 : 90% of the energy needs **119 modes**, 99% needs **224**, close to one mode per snapshot
- For a frame in the training set, 100 modes still leave 31% error, and bubbles and thin plumes are smeared
- The structures move, while each POD mode is a fixed pattern in space

POD is the optimal linear basis, and moving structures still need many modes.

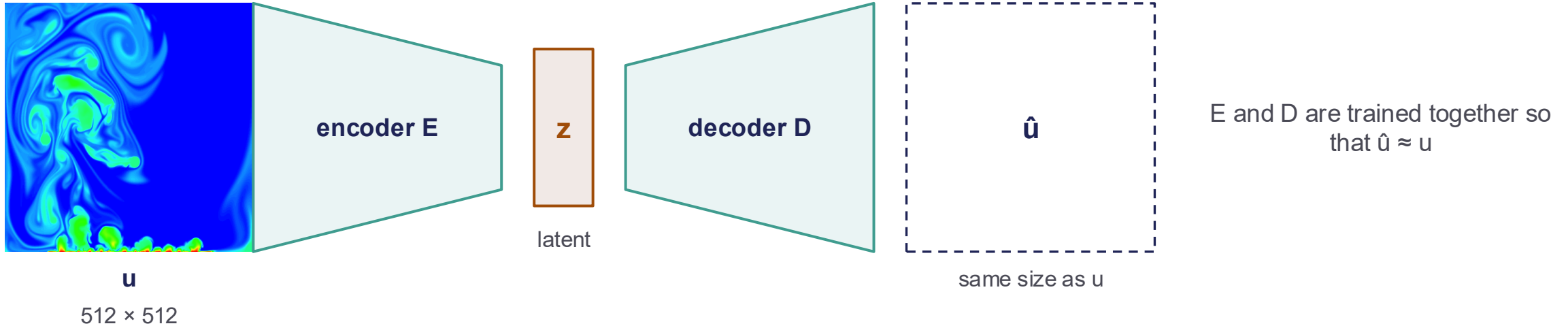
Moving fronts need many POD modes



- One shape carried across the domain: each snapshot is simple, but each position needs different modes
- 99% of the energy needs **41 modes**. With 20 modes the error is 46% and the pulse ripples; here the POD modes are Fourier modes, so these are the ripples from Lecture 3 when truncating a Fourier series

The pulse has one degree of freedom, its position, and it enters nonlinearly.

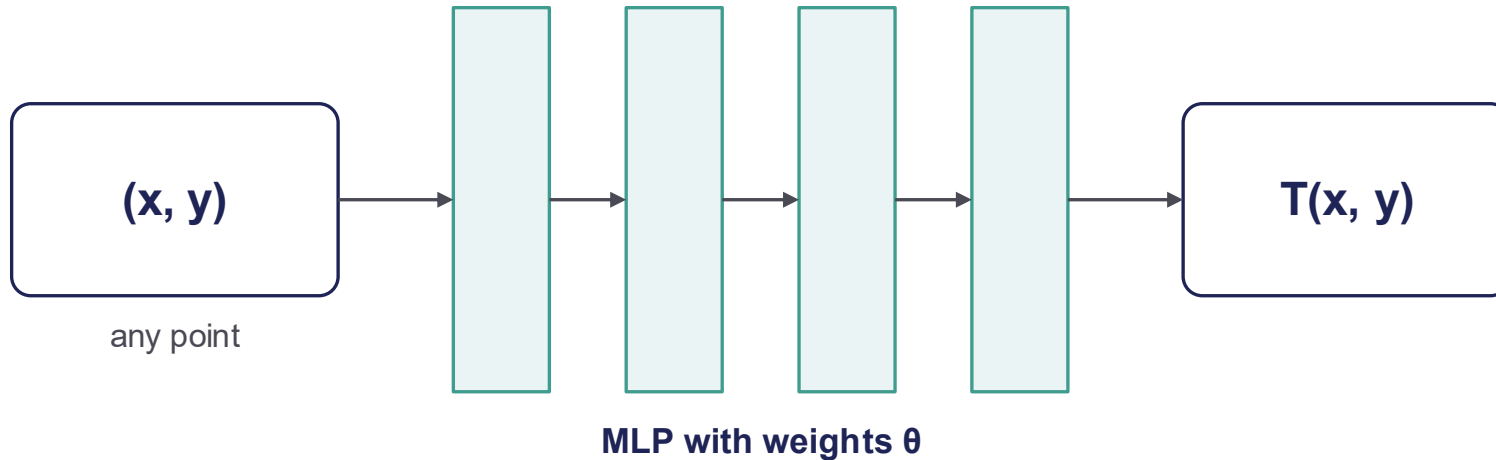
Autoencoders and learned latent representations



$$z = E(u), \quad \hat{u} = D(z), \quad \min \|D(E(u)) - u\|^2$$

- An **autoencoder**: an encoder network maps the field to a vector z , the **latent**; a decoder maps it back. Trained to reconstruct its input (**self-supervised**)
- With a nonlinear decoder, a moving pulse can be described by its position: one latent number
- With linear E and D and MSE loss, the best autoencoder spans the same subspace as POD (Baldi & Hornik 1989)
- Used to compress fields before learning dynamics or generating new samples: latent diffusion, foundation models

Neural fields represent a field as a function

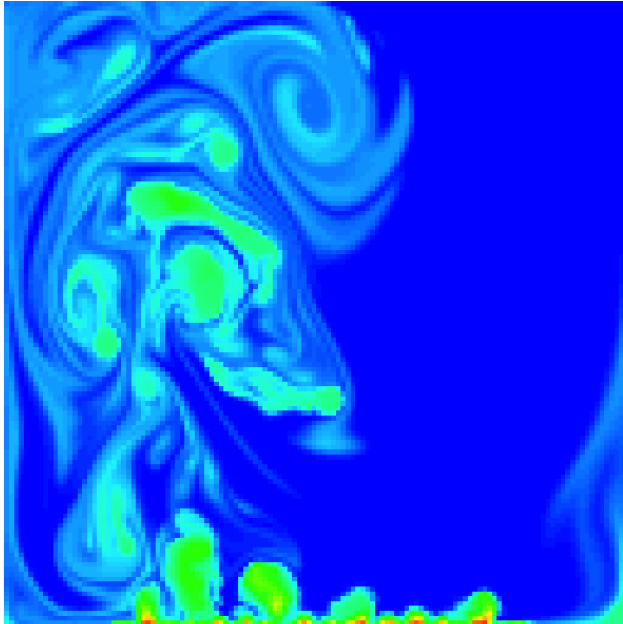


$$u_{\theta} : (x, y, t) \mapsto u$$

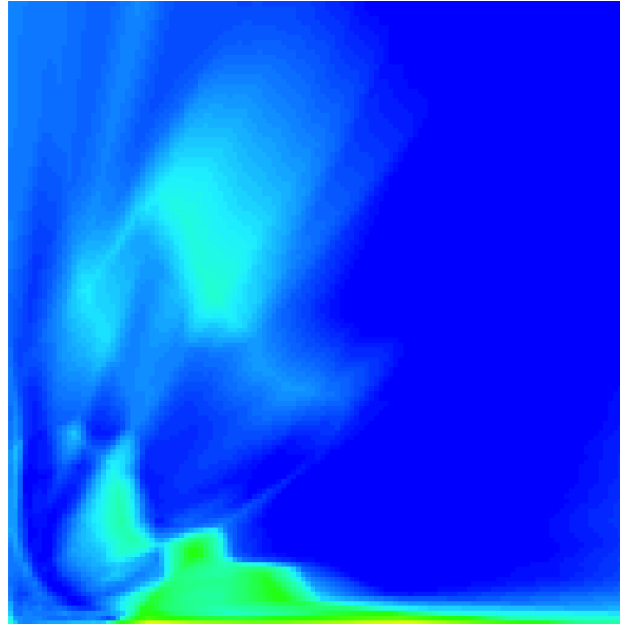
- A network takes coordinates as input and returns the field value at those locations. Its **weights θ** are the stored representation
- It can be evaluated at **any point** and at **any resolution**
- Derivatives come from automatic differentiation
- A PINN (Lecture 13) is a neural field trained to satisfy the PDE; operator learning (Lecture 11) maps functions to functions
- Each field requires training a network, and the weights can outnumber the grid values

Spectral bias in neural fields

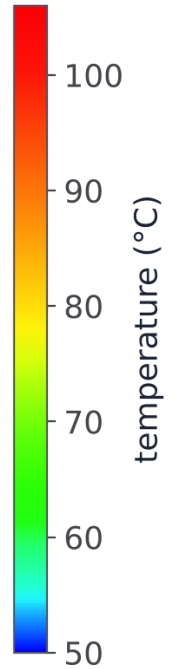
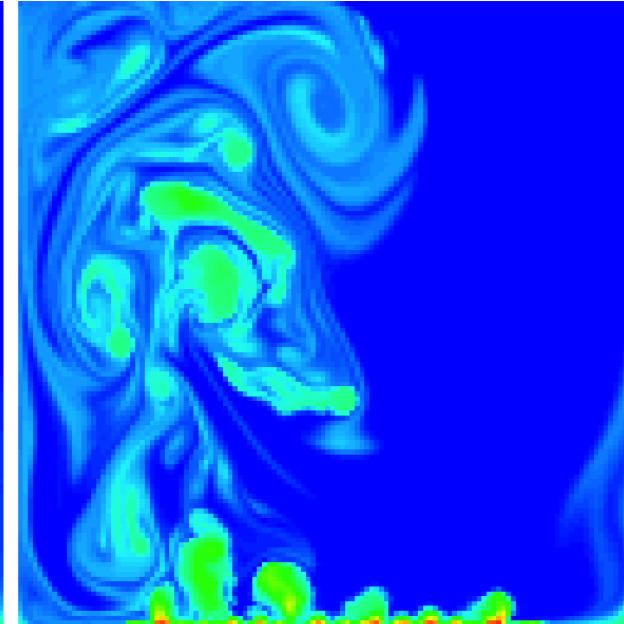
target: 128 × 128 frame
16,384 values



MLP on (x, y)
error 51%, 198,401 weights



MLP on Fourier features of (x, y)
error 0.1%, 263,425 weights



- With (x, y) as input, the MLP captures the large-scale shape and blurs the plumes. This is **spectral bias**: networks fit low wavenumbers first
- Mapping (x, y) to sines and cosines of random frequencies first removes the blur (Tancik et al. 2020). SIREN uses sine activations instead
 - both: 4 hidden layers x 256 neurons with ReLU activation, Adam (lr 10^{-3}), 3000 steps of 4096 random pixels

$$\gamma(\mathbf{x}) = [\sin(2\pi B\mathbf{x}), \cos(2\pi B\mathbf{x})], \quad B_{ij} \sim \mathcal{N}(0, \sigma_B^2)$$

B: 128 x 2 matrix of random frequencies, $\sigma_B = 10$

Comparing representations

Representation	What is stored	Strengths	Limitations	Typical model
Regular grid	values on $H \times W$ cells	neighbors by index; fast convolution	resolution fixed; curved walls become staircases	CNN, U-Net
Mesh (graph)	node features, positions, edge list	complex geometry; local refinement	irregular layout	graph network
Point cloud	positions and features	moving points at arbitrary positions	neighbors found by distance; output must be independent of row order	particle or point network
Basis coefficients	r coefficients (Fourier, POD)	smooth or typical structure in fewer values	moving and sharp features need many coefficients	linear models; Fourier layers
Learned latent	short vector z	nonlinear structure in fewer values	Latent has no clear meaning; needs a trained decoder	latent dynamics, diffusion
Neural field	network weights θ	any resolution; exact derivatives	slow to fit; spectral bias	PINN, neural operator

Each representation determines which structure the model has built in and which it must learn from data.

Predicting the next state or the change

- **Recall:** Forward Euler adds a small increment

$$u^{n+1} = u^n + \Delta t \alpha L u^n$$

- L: the stencil as a matrix; Δt : time step; α : diffusivity
- SmallCNN (Lecture 1) has the same form: return $x + \text{self.net}(x)$

$$\hat{u}_{n+1} = u_n + f(u_n; \theta)$$

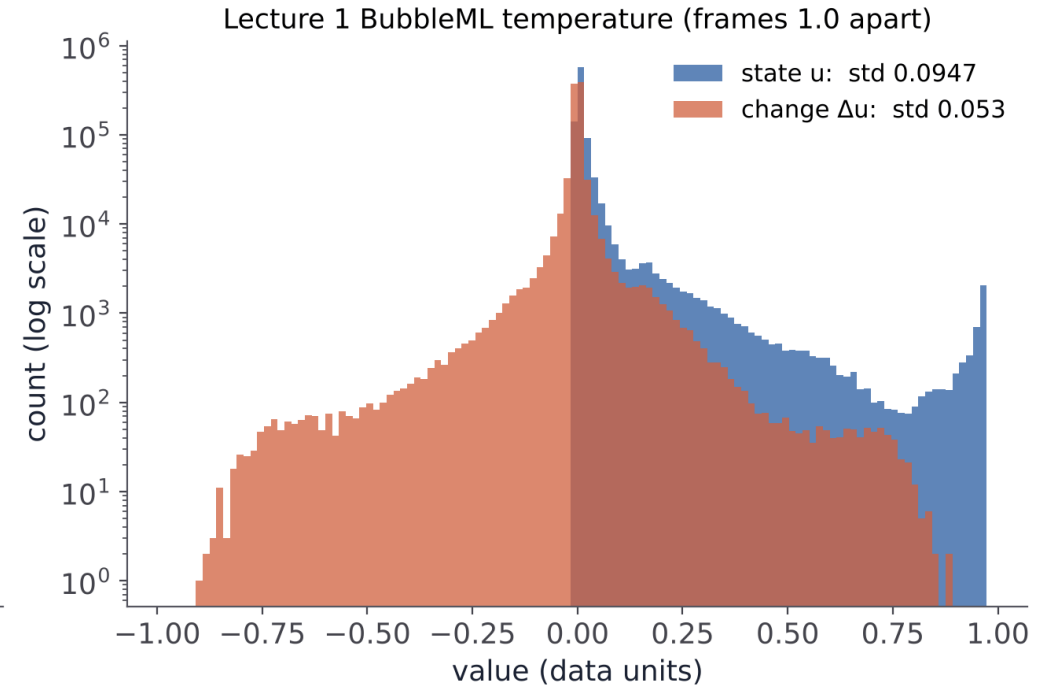
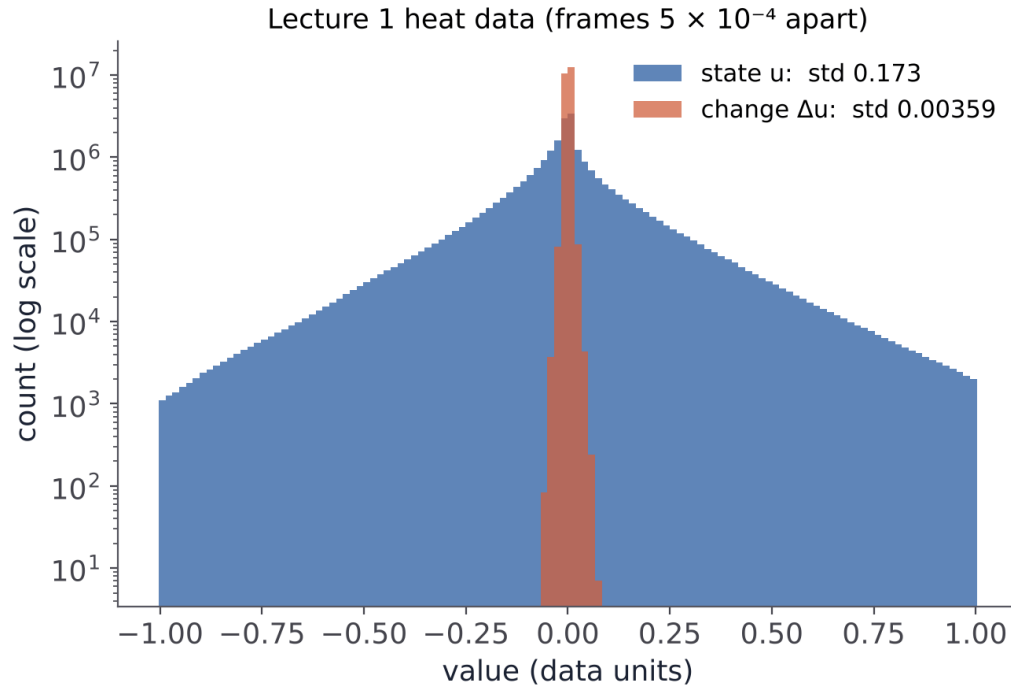
- So MSE on u_{n+1} equals MSE on the increment $\Delta u = u_{n+1} - u_n$:

$$\|u_n + f(u_n; \theta) - u_{n+1}\|^2 = \|f(u_n; \theta) - \Delta u\|^2$$

- Both forms can **represent the same maps**; they differ in what the network outputs
- In the residual form, an untrained network outputs small random values, so the model starts close to “no change”, a good starting point for small time steps. Otherwise the network first has to learn to copy its input
- The scale of Δu can be much smaller than u

Residual prediction is forward Euler with a learned right-hand side.

Normalizing the increment



- Std of Δu relative to std of u : **2%** for the heat data and **56%** for the BubbleML file, where consecutive frames differ substantially
- **Standardize the target Δu** with its mean and standard deviation from the training set, so the network outputs values of order 1

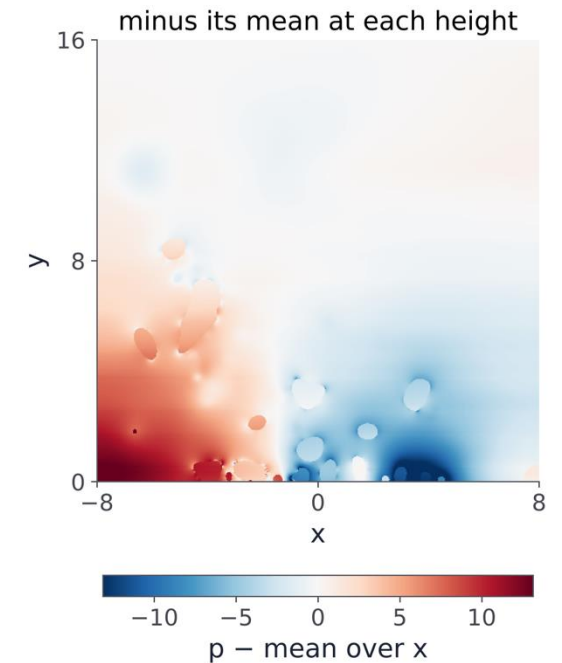
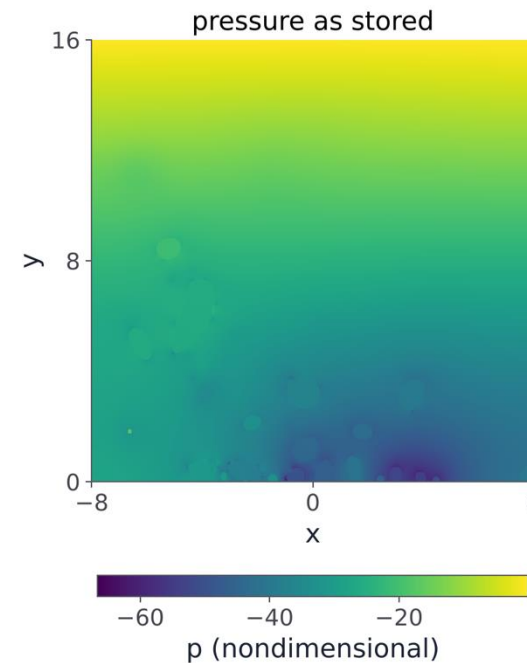
$$\hat{u}_{n+1} = u_n + \mu_{\Delta} + \sigma_{\Delta} f(\tilde{u}_n; \theta), \quad \mu_{\Delta}, \sigma_{\Delta} \text{ from } \Delta u = u_{n+1} - u_n$$

Physics-based normalization

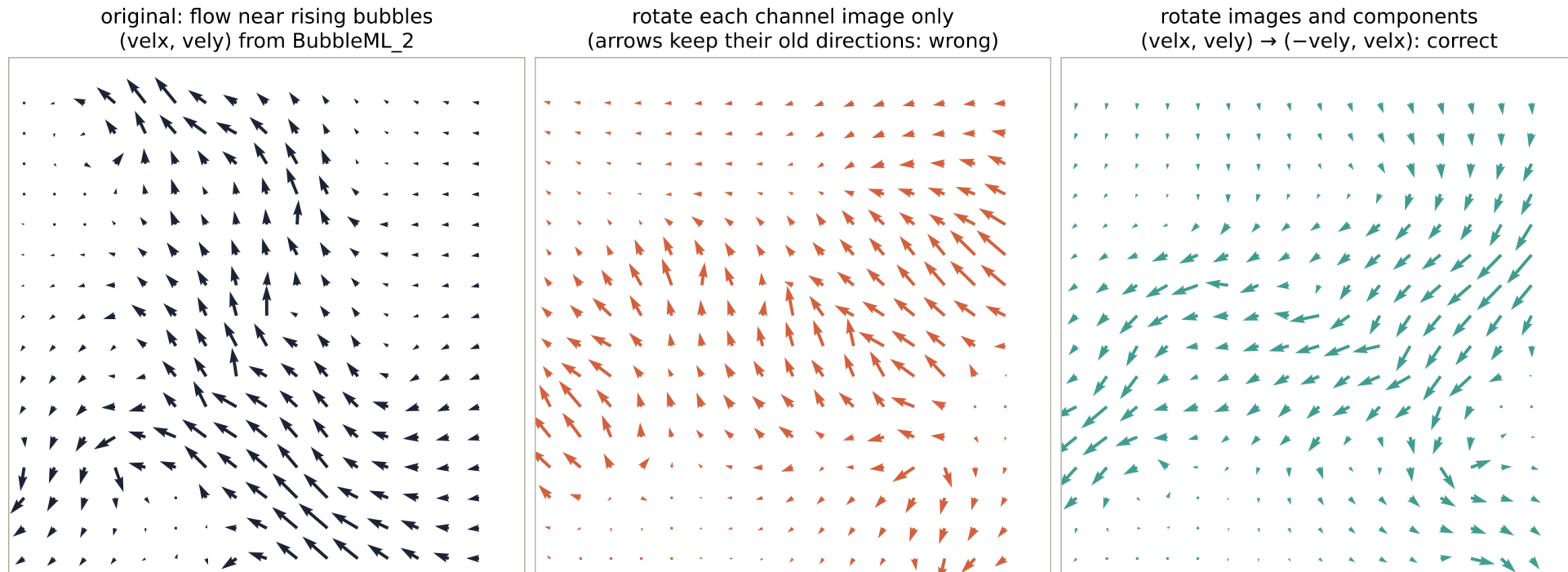
- **Nondimensional variables** remove units.
- BubbleML_2 stores **temperature** in °C. Dividing by the heater-liquid temperature difference gives a **nondimensional temperature** θ : 0 in the bulk liquid, 1 at the heater, for every T_{wall} (the same idea as Re and Pe in Lecture 3):

$$\theta = \frac{T - T_{\text{bulk}}}{T_{\text{wall}} - T_{\text{bulk}}}$$

- **Pressure** enters the equations only through its gradient, so a constant offset leaves the flow unchanged: subtract a reference value
- The stored pressure is mostly a **vertical gradient**, the weight of the liquid above (−66 at the bottom to 0 at the top). Subtracting the mean at each height removes this gradient; the variations due to the flow and bubbles remain
- For positive quantities that span orders of magnitude (density, concentration), model $\log u$



Velocity components rotate together



- Rotating the domain by 90° moves a scalar field (temperature, pressure, dfun). A vector field also rotates, so its components mix:

$$T'(\mathbf{x}) = T(R^{-1}\mathbf{x}), \quad \mathbf{v}'(\mathbf{x}) = R \mathbf{v}(R^{-1}\mathbf{x}), \quad R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$$

- Rotating velx and vely as two independent images gives a wrong field, for example in **data augmentation**.

Summary

- A stored value can be a point value, a cell average or a coefficient
- Grid inputs can include parameters, coordinates and geometry as channels; padding is a boundary condition
- Resolution and frame spacing determine what the model sees; average before downsampling to limit aliasing
- Meshes, point clouds and spherical meshes represent geometry that regular grids handle poorly, and call for graph or point-based models
- POD gives the optimal linear basis for a dataset; moving sharp features need many modes, which motivates learned latents and neural fields
- Predicting the increment is forward Euler with a learned update; normalize the target with its own statistics or with physical scales
- Vector components transform together under rotation

After class

- **Brunton & Kutz, *Data-Driven Science and Engineering***, 2nd ed. (Cambridge, 2022)
 - Ch. 1: the SVD, with POD and its Eckart-Young optimality
 - Ch. 2: Fourier transforms and sampling
- **Thuerey et al., *Physics-based Deep Learning*** (physicsbaseddeeplearning.org)
 - chapters on supervised training and on graph-based and unstructured data
- **Prince, *Understanding Deep Learning*** (udlbook.github.io)
 - Ch. 10: convolutional networks and padding · Ch. 13: graph neural networks · Ch. 17: autoencoders
- **Tancik et al., Fourier features demo** (bmild.github.io/fourfeat), an interactive view of spectral bias

Next, Lecture 5 (Thu, Oct 8): symmetry, invariance and equivariance

NEXT: HANDS-ON

From concepts to code

→ [Open the Lecture 4 notebook](#)

1. Downsample a BubbleML frame by sampling and by averaging; compare their spectra
2. Compute POD for the heat data and BubbleML; reconstruct with $r = 5, 20, 50$
3. Retrain SmallCNN on the heat data with a standardized increment target; compare rollouts with Lecture 1